

UNIVERSIDAD DE LAS AMÉRICAS

CAMPUS: UDLAPARK



CARRERA: INGENIERÍA EN SOFTWARE

ASIGNATURA: Ing. Web

MODALIDAD: Presencial

TEMA: Aplicación mejores prácticas Principios Solid y patrones de diseño

AUTOR: ROBERTO MANOSALVAS

MARZO 2026 – JULIO 2026

QUITO – ECUADOR

Identificación del problema dentro de las restricciones del proyecto

El repositorio base ya contiene intentos de varios patrones de diseño (Repository, Singleton, Factory Method y Builder), pero todos están aplicados de forma incompleta o inconsistente. A continuación se describe cada problema, su causa técnica y el principio SOLID que vulnera.

Inconsistencia en el patrón Repository (reporte de QA)

HomeController recibe IVehicleRepository inyectado por constructor y lo usa correctamente en AddMustang() y AddExplorer() (_vehicleRepository.AddVehicle(vehicle)). Sin embargo, el método Index() no usa esa misma abstracción: lee directamente VehicleCollection.Instance.Vehicles, que es un Singleton estático manual definido en Infraestructure/Singletons/VehicleCollection.cs.

Esto es precisamente el bug que reporta QA: el alta de vehículos pasa por el repositorio, pero la lectura para listar vehículos en pantalla pasa por otra vía (el Singleton). Hoy el resultado visual coincide porque MyVehiclesRepository también envuelve ese mismo Singleton, pero el diseño es frágil: el día que se registre otra implementación de IVehicleRepository (por ejemplo, una para pruebas que no use el Singleton), Index() seguiría mostrando los datos del Singleton en lugar de los del repositorio real.

Principio violado: Dependency Inversion Principle (DIP). El controlador depende de una clase concreta y estática (VehicleCollection) en vez de depender únicamente de la abstracción IVehicleRepository que ya tiene inyectada.

Imposibilidad de probar sin base de datos

DBVehicleRepository implementa IVehicleRepository pero los tres métodos lanzan NotImplementedException. Esto es esperado (el esquema de BD no está listo), pero revela una segunda limitación: la única alternativa de prueba que existe hoy (MyVehiclesRepository) depende de un Singleton manual con estado estático global (VehicleCollection._instance), lo cual:

- Impide aislar pruebas unitarias entre sí (el estado persiste entre tests porque vive en un campo static).
- Duplica una responsabilidad que el propio contenedor de Inyección de Dependencias (DI) de ASP.NET Core ya resuelve mejor con un ciclo de vida Singleton administrado.

Principio/práctica afectada: uso del patrón Singleton como anti-patrón (estado global mutable, oculto, no inyectable) en lugar de delegar la vida única de la instancia al contenedor de DI.

Acoplamiento en la creación de vehículos y falta de extensibilidad

AddMustang() y AddExplorer() instancian directamente new FordMustangCreator() y new FordExplorerCreator() dentro del controlador. El patrón Factory Method (clase abstracta Creator) ya existe, pero el controlador no depende de esa abstracción: la usa de forma concreta. Como consecuencia, cada modelo nuevo (como el Ford Escape que pide el negocio) obliga a:

1. Crear la clase Creator concreta (correcto, esperado).
2. Modificar HomeController para agregar una acción nueva (innecesario).
3. Modificar la vista Index.cshtml para agregar el enlace (innecesario).

Principios violados: DIP (el controlador depende de clases concretas Creator) y Open/Closed Principle — OCP (agregar un modelo obliga a modificar código existente en vez de solo extenderlo).

Falta de un mecanismo escalable para propiedades por defecto

CarBuilder es un intento de Builder, pero sus valores por defecto (Brand="Ford", Model="Mustang", Color="Red") están hardcoded en una clase que debería ser genérica para cualquier Car. Esa responsabilidad —decidir qué modelo y color por defecto tiene un Ford Mustang— le corresponde al Creator, no al Builder. Además, si negocio agrega el año actual y otras ~20 propiedades el próximo sprint, hacerlo como parámetros de constructor de Vehicle obligaría a modificar Vehicle, Car, Motorcycle y todos los Creators existentes.

Principios violados: Single Responsibility Principle — SRP (el Builder mezcla defaults de un modelo específico con la lógica genérica de construcción) y OCP (cada propiedad nueva del próximo sprint obligaría a tocar clases ya estables).

Restricciones del proyecto

- El esquema de base de datos no está disponible: cualquier solución debe funcionar 100% en memoria hoy y permitir swap a persistencia real sin tocar el controlador ni la vista.
- Negocio anticipa ~20 propiedades adicionales sobre Vehicle en el siguiente sprint: la solución debe minimizar el código a tocar cuando eso ocurra.
- El Arquitecto de Software ya fijó la dirección técnica para nuevos modelos: Factory Method.
- Trabajo individual, código debe quedar comentado y subido a un repositorio Git personal.

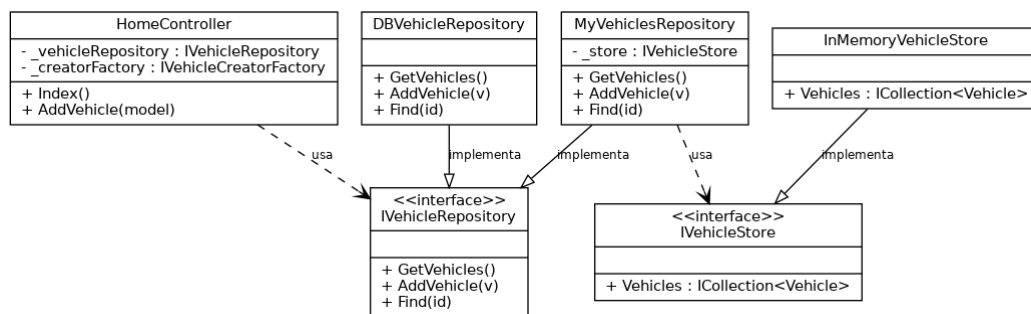
Selección de patrones y metodologías para solucionar el problema

Se proponen tres patrones, cada uno resolviendo un problema puntual de la sección 1, más un ajuste de buenas prácticas en el uso de Inyección de Dependencias para sustituir el Singleton manual.

Repository Pattern (corregido) + Singleton administrado por el contenedor de DI

Problema que resuelve

Se mantiene `IVehicleRepository` como única fuente de verdad: tanto la lectura (`Index`) como la escritura (`AddVehicle`) pasan por la misma abstracción inyectada. El estado en memoria se extrae a una nueva abstracción, `IVehicleStore`, registrada con `services.AddSingleton<IVehicleStore, InMemoryVehicleStore>()` en el contenedor de DI, eliminando el Singleton estático manual (`VehicleCollection`).

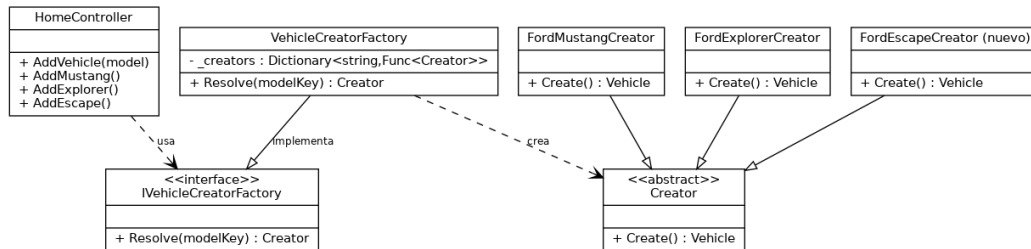


Justificación técnica: con este diseño, cuando el equipo de BD entregue el esquema, el único cambio necesario es una línea en `ServicesConfiguration.cs` (registrar `DBVehicleRepository` en vez de `MyVehiclesRepository`). Ni `HomeController` ni las vistas se modifican, porque ambos dependen de la interfaz, no de la implementación (cumple DIP). Además, el nuevo `IVehicleStore` puede sustituirse por una instancia limpia en cada prueba unitaria, resolviendo el pedido del equipo de BD de "probar sin guardar en base de datos".

Factory Method con resolución por nombre de modelo

Problema que resuelve: el requerimiento explícito del Arquitecto de Software (nuevo modelo Ford Escape).

Se formaliza la jerarquía `Creator` (clase abstracta) → `FordMustangCreator`, `FordExplorerCreator`, `FordEscapeCreator` (nuevo). El controlador deja de instanciar estas clases directamente: en su lugar depende de `IVehicleCreatorFactory`, una abstracción que resuelve el `Creator` correcto a partir de un nombre de modelo ("Mustang", "Explorer", "Escape"), usando un diccionario registrado en `VehicleCreatorFactory`.

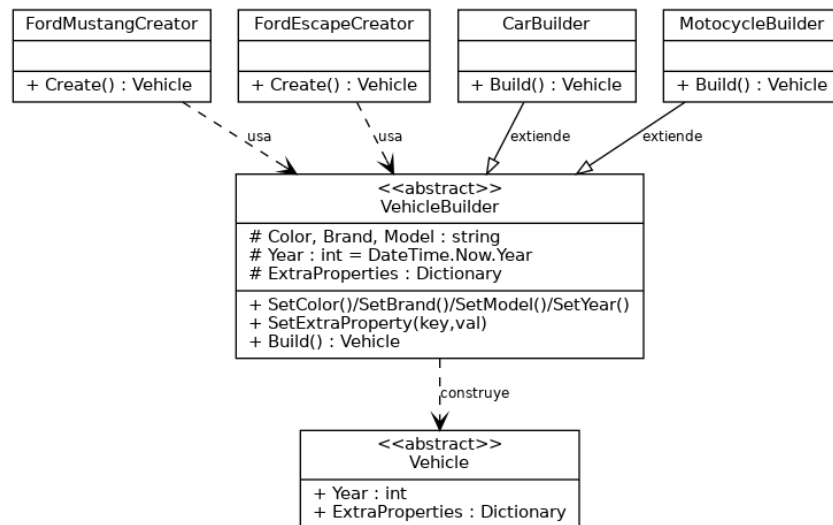


Justificación técnica: agregar el modelo Escape (Color: Red, Marca: Ford, Modelo: Escape) implicó únicamente crear FordEscapeCreator.cs y añadir una entrada en el diccionario de VehicleCreatorFactory. No se modificó Creator, ni IVehicleRepository, ni la firma de ningún método existente (cumple OCP). El endpoint genérico AddVehicle(string model) del controlador queda preparado para cualquier modelo futuro sin volver a tocar el controlador; se conservan además AddMustang/AddExplorer/AddEscape como atajos finos sobre ese endpoint, por requerimiento explícito del enunciado.

Builder Pattern con defaults centralizados

Problema que resuelve: (año actual + ~20 propiedades del siguiente sprint).

Se introduce VehicleBuilder, una clase abstracta que centraliza los valores por defecto comunes a cualquier vehículo: Color, Brand, Model, Year (= DateTime.Now.Year) y un diccionario ExtraProperties pensado específicamente para las propiedades que negocio aún no ha definido. CarBuilder y MotorcycleBuilder extienden VehicleBuilder y solo agregan el Build() específico de su tipo concreto; ya no contienen defaults de un modelo particular.



Justificación técnica — minimización de cambios para el siguiente sprint: cuando negocio defina las 20 propiedades reales, el trabajo se reduce a un solo archivo (VehicleBuilder.cs):

agregar el campo con su valor por defecto y, si se necesita, un método fluido SetX(). No es necesario tocar Vehicle, Car, Motorcycle ni ningún Creator existente, porque estos no participan en la definición de los defaults. Esto es exactamente lo que pide el enunciado: "diseñarlo para minimizar los cambios para el siguiente sprint".

Resumen de relación problema → patrón → principio SOLID reforzado

Problema (sección 1)	Patrón aplicado	Principio SOLID reforzado
1.1 Index() bypasa el repositorio	Repository Pattern (uso consistente)	DIP
1.2 No se puede probar sin BD	DI Singleton (IVehicleStore) en vez de Singleton manual	DIP, testabilidad
1.3 Alta de vehículos acoplada a clases concretas	Factory Method + resolver por nombre	DIP, OCP
1.4 Defaults hardcodeados y constructor rígido	Builder Pattern (defaults centralizados)	SRP, OCP

Propuesta técnica y prototipo de la solución

Esta sección resume los cambios de código aplicados sobre el repositorio base (juanjoleong/Best-Practices-Udla-Workshop), organizados por patrón. El código completo, comentado, queda disponible en el repositorio Git personal indicado en la portada.

Cambios para el Repository Pattern

- Se eliminó Infraestructure/Singletons/VehicleCollection.cs (Singleton manual).
- Se agregó Infraestructure/Storage/IVehicleStore.cs e InMemoryVehicleStore.cs.
- MyVehiclesRepository ahora recibe IVehicleStore por constructor (antes llamaba a VehicleCollection.Instance).
- HomeController.Index() ahora usa _vehicleRepository.GetVehicles() en vez de VehicleCollection.Instance.Vehicles.
- ServicesConfiguration registra: AddSingleton<IVehicleStore, InMemoryVehicleStore>() y AddTransient<IVehicleRepository, MyVehiclesRepository>().

Fragmento relevante (HomeController.cs):

```
public IActionResult Index()
{
    var model = new HomeViewModel();

    // BUGFIX: antes leia VehicleCollection.Instance.Vehicles directamente.

    model.Vehicles = _vehicleRepository.GetVehicles();

    ...
}
```

Cambios para el Factory Method

- Se agregó Infraestructure/Factories/IVehicleCreatorFactory.cs y VehicleCreatorFactory.cs (resolver por nombre de modelo).
- Se agregó Infraestructure/Factories/FordEscapeCreator.cs (Color: Red, Brand: Ford, Model: Escape).
- HomeController recibe IVehicleCreatorFactory por constructor; se agregó la acción genérica AddVehicle(string model) y AddMustang/AddExplorer/AddEscape quedaron como atajos de una línea.
- Se agregó el enlace "Add Escape" en Views/Home/Index.cshtml.

Fragmento relevante (FordEscapeCreator.cs):

```
public class FordEscapeCreator : Creator
{
    public override Vehicle Create()
    {
        return new CarBuilder()
            .SetBrand("Ford")
            .SetModel("Escape")
            .SetColor("Red")
            .Build();
    }
}
```

Cambios para el Builder Pattern

- Se agregó ModelBuilders/VehicleBuilder.cs (abstracta, con defaults centralizados: Color, Brand, Model, Year, ExtraProperties).
- CarBuilder ahora extiende VehicleBuilder y ya no hardcodea "Ford"/"Mustang"/"Red" (esos defaults se movieron a FordMustangCreator, que es quien realmente los conoce).
- Se agregó ModelBuilders/MotocycleBuilder.cs, disponible para cuando negocio pida modelos de motocicleta.
- Vehicle.cs ganó dos propiedades sin tocar su constructor: Year (int) y ExtraProperties (Dictionary<string, object>), este último reservado para las ~20 propiedades del siguiente sprint.

Fragmento relevante (VehicleBuilder.cs):

```
protected int Year = DateTime.Now.Year;
protected Dictionary<string, object> ExtraProperties = new();

public VehicleBuilder SetExtraProperty(string key, object value)
{
    ExtraProperties[key] = value;
    return this;
}
```

Cómo minimiza esto el trabajo del siguiente sprint

Cuando negocio entregue la lista real de las 20 propiedades, el flujo de trabajo es:

1. Abrir únicamente VehicleBuilder.cs.
2. Agregar el campo con su valor por defecto (o una entrada en ExtraProperties si no requiere tipado fuerte de inmediato).
3. Opcionalmente, agregar un método fluido SetX() si algún Creator necesita sobrescribir ese default.

No se requiere recompilar ni modificar Vehicle, Car, Motocycle, ningún Creator existente, el HomeController ni las vistas — el cambio queda contenido en un solo archivo, que es el objetivo central del patrón Builder aplicado aquí.

Conclusiones

- El bug reportado por QA se origina en una violación de DIP (Index() no usaba la abstracción IVehicleRepository); se corrigió unificando toda lectura/escritura de vehículos a través del repositorio.
- La necesidad de probar sin base de datos se resuelve formalizando el Repository Pattern y sustituyendo el Singleton manual por un Singleton administrado por el contenedor de DI (IVehicleStore), lo que además mejora la testabilidad.
- El Factory Method, ya sugerido por el Arquitecto de Software, se completó con un resolver por nombre de modelo, de forma que agregar el Ford Escape (y cualquier modelo futuro) sea una extensión y no una modificación del código existente (OCP).
- El Builder Pattern, generalizado y con defaults centralizados, resuelve el pedido de negocio (año actual + 20 propiedades futuras) minimizando el alcance de los cambios del siguiente sprint a un único archivo.

En conjunto, las tres correcciones devuelven el código a un estado consistente con los principios SOLID que el patrón original buscaba aplicar, sin alterar el comportamiento visible para el usuario final de la aplicación.